Soft-Orthogonal Trust Region (SOTR) – “Ortho” as a Controllable Constraint.
Problem: Muon’s full orthonormalization is powerful but can be too rigid or costly. We want to enforce orthogonality softly, improving conditioning while not completely overriding gradient information, and ensure updates stay in a “trust region” around the current weights. This aims to prevent runaway weight growth or ill-conditioned updates, especially in very deep nets, without fully discarding magnitude information.
Method Overview: SOTR modifies Muon-style updates with a tunable orthogonality strength. For each weight gradient matrix G, we compute a partial polar update: let $O_\text{approx} = \text{NewtonSchulz}_p(G)$ with p fewer iterations than Muon (e.g. 1–2 iterations instead of 5) – this yields a matrix that is closer to orthonormal but not fully (singular values moved closer to 1, but not all equal)�. Alternatively, compute full $O = UV^T$ and then blend it with the original gradient: $U = \alpha \cdot O + (1-\alpha)\cdot \frac{G}{|G|_F}$, where $\alpha \in [0,1]$ is a hyperparameter controlling “how orthonormal” the update is. We also enforce a trust-region radius: ensure $|U|_F \le \Delta$, a chosen threshold (if not, scale U down). This Δ effectively caps the update size like global clipping but on the matrix as a whole. Then apply momentum etc. In code: for each 2D weight:
kellerjordan.github.io
Copy code
Python

```
G = grad(W)
O = orthogonalize(G)        # e.g., using 1-2 Newton-Schulz iters or SVD truncated
U = alpha * O + (1-alpha) * (G / (eps + fro_norm(G)))  # blend with normalized grad
if fro_norm(U) > Delta:
    U = U * (Delta / fro_norm(U))    # trust-region clip
momentum_W = beta1 * momentum_W + (1-beta1) * U   # momentum update
W = W - lr * momentum_W   # weight update
```

We choose α (say 0.5 initially) and trust radius

Δ (maybe calibrated to layer dimensions or a constant like 1.0). Why it might work: This gives us the conditioning benefits of orthonormalization (since U is partly orthogonalized, boosting weaker directions) but still retains some magnitude info from G (so if G had an overall scale or if one singular value is legitimately more important, we don't ignore it completely). The trust region Δ ensures no update is too large in Frobenius norm – a safety similar to global grad clipping but per matrix. Geometrically, we're constraining updates to stay within a certain distance, and nearer to the orthonormal manifold without completely projecting onto it. This could avoid overshooting minima and allow smoother convergence. Also, by adjusting α, we can fine-tune implicit regularization: $\alpha=1$ reduces to Muon (full orthonorm, strong reg), $\alpha=0$ reduces to just normalized gradient (like sign of matrix). Intermediate α is new territory – e.g., $\alpha=0.5$ might yield best of both worlds. Predicted Trade-offs: If α is too high (almost Muon), we could still encounter overhead and the risk of under-utilizing gradient magnitudes (maybe slower in early training if large singular directions carry signal). If α too low, we don't fix conditioning enough. We'll tune α on a small model first. The trust radius Δ might sometimes clip updates, effectively lowering LR for large updates, which could slow convergence a bit but improve stability (a conscious trade-off). Memory overhead is minimal (just storing a matrix copy for O and computing NS a couple of times). Compute overhead: one or two matrix multiplications for NS – with p=1 or 2, that's way cheaper than Muon's p=5 and likely negligible relative to a forward/backward pass on large model. Implementation Plan: We integrate this into an optimizer class in PyTorch (possibly building atop the existing Muon code by altering iteration count and adding blending). We'll make α and Δ hyperparameters, possibly schedule α over time (e.g., start with smaller α early when gradients are informative, increase α later to encourage orthonormalization as training progresses and conditioning issues mount). The algorithm can leverage FSDP (FullyShardedDataParallel) by performing the partial orthonormalization on local shards – similar to MuonBP,

we can do it block-wise and do periodic full sync if needed (maybe not necessary if $\alpha<1$, as we're not forcing full consistency across shards). Validation Plan:
Small-scale speedrun: Use the NanoGPT or FinetuneGPT setup and replicate a speedrun to a given loss as in Muon's blog. Compare SOTR with $\alpha=0.5, 0.7, 1.0$ (Muon) vs AdamW vs Muon. Metrics: steps and time to reach target loss, and condition number of weight gradients during training (we can monitor singular values on a small layer as proxy). We expect SOTR with e.g. $\alpha=0.5$ to approach target loss maybe slightly slower than $\alpha=1$ Muon but faster than AdamW, and with fewer stability issues (maybe fewer spikes in grad norm).
Mid-scale test (300M–1B): Train a ~400M transformer on The Pile subset for a few epochs with SOTR vs AdamW vs Muon. Use same LR schedule. Evaluate final perplexity. We anticipate SOTR matching Muon's perplexity (since orthonormalization seems not to hurt quality) but with easier hyperparam tuning (Muon needed weight decay and scale tweaks; SOTR might be more plug-and-play since it's gentler). Also track training stability: any divergence or need for clipping? The trust-region likely prevents divergence entirely.
Ablations: Try without trust radius (just α blend) to see if trust region was triggering and helpful. Also try α schedule: e.g. increase from 0 to 0.5 over first 10k steps to gradually introduce orthonorm.
Distributed run: Using FSDP, train a ~1B model across 8 GPUs with SOTR. Check throughput overhead vs AdamW: expecting maybe <5% overhead (due to a couple of extra matmuls). This confirms scalability. The outcome of SOTR would be an optimizer that one can tune via a single parameter α to dial between SGD-like and Muon-like behavior, giving more flexibility and possibly better performance in the middle. Success is measured by if α between 0 and 1 yields faster or more stable training than either extreme. If yes, SOTR could be a new standard approach to reap Muon's benefits without all its quirks.

Turn the above into a LLM prompt for Claude Code to design, perform and validate the idea proposed therein.

Below is a **ready-to-use, copy-paste LLM prompt** tailored for **Claude Code** (or any strong coding-capable LLM) to **design, implement, experiment with, and validate Soft-Orthogonal Trust Region (SOTR)** exactly as you described.
It is scoped to elicit **concrete code, ablations, metrics, and interpretation**, not just high-level commentary.

🔧 Claude Code Research & Engineering Prompt

You are a **research engineer and optimizer designer**. Your task is to **design, implement, and empirically validate a novel optimizer called Soft-Orthogonal Trust Region (SOTR)** for training transformer models.

This project builds directly on recent work on **Muon-style orthogonalized optimizers**, but introduces **controllable, soft orthogonality and an explicit trust-region constraint** to improve conditioning, stability, and practicality.

---

1. Problem Context

Muon-style optimizers enforce **full orthonormalization of gradient matrices**, which improves conditioning but can be:

- Too rigid (discarding magnitude information),
- Computationally costly,
- Sensitive to hyperparameters in early training.

We propose **Soft-Orthogonal Trust Region (SOTR)**, which:

- Enforces orthogonality **softly**, not exactly,
- Preserves some gradient magnitude information,
- Constrains updates to a **matrix-level trust region** (Frobenius norm cap),
- Interpolates smoothly between Adam/normalized-gradient behavior and full Muon.

Your goal is to **implement, test, and analyze SOTR rigorously**.

---

2. Algorithm Specification (Must Be Followed Exactly)

For each 2D weight matrix W with gradient G:

Step 1: Partial Orthogonalization

Compute an *approximate orthogonal factor* using **either**:

- **Partial Newton–Schulz**:
  - Run **p = 1–2 iterations only** (not full Muon)
  - Output O_{approx}

OR

- **Exact SVD**:
  - $G = U\Sigma V^T$, then $O = UV^T$
  - Used only for ablations / correctness checks

Step 2: Soft Orthogonal Blending

Blend orthogonalized direction with normalized raw gradient:

$$U = \alpha \cdot O + (1 - \alpha) \cdot \frac{G}{\|G\|_F + \epsilon}$$

- $\alpha \in [0, 1]$ is a tunable hyperparameter
- $\alpha = 1 \Rightarrow$ Muon
- $\alpha = 0 \Rightarrow$ normalized-gradient update

Step 3: Trust Region Enforcement

Apply a **matrix-level trust region**:

$$\text{if } \|U\|_F > \Delta: \quad U \leftarrow U \cdot \frac{\Delta}{\|U\|_F}$$

- Δ is the trust-region radius
- This is **not elementwise clipping**, but whole-matrix constraint

Step 4: Momentum & Update

Apply momentum and weight update:

```python
momentum = beta1 * momentum + (1 - beta1) * U
W = W - lr * momentum
```

- Use Adam-style momentum (no adaptive second moment initially)
- Weight decay optional (test with and without)

---

3. Required Implementation Tasks

A. Optimizer Code

- Implement `SOTROptimizer` in PyTorch
- Support:
  - `alpha`, `Delta`, `beta1`, `lr`
  - `p` Newton–Schulz iterations
  - Layer-wise handling of 2D parameters only
- Clean, modular code suitable for open-sourcing

B. Efficiency

- Measure per-step overhead vs AdamW and Muon
- Ensure:
  - No unnecessary SVDs in main experiments
  - NS iterations fused efficiently
- Target: **<5% overhead** vs AdamW

---

4. Experimental Validation Plan (You Must Execute This)

4.1 Small-Scale Speedrun

Setup

- Model: NanoGPT or similar (~10–50M)
- Dataset: Tiny Shakespeare or OpenWebText subset

Compare

- AdamW
- Muon (full orthonorm)
- SOTR with:
  - $\alpha \in \{0.3, 0.5, 0.7, 1.0\}$
  - Fixed $\Delta = 1.0$

Metrics

- Steps & wall-clock time to reach target loss
- Gradient Frobenius norm over time
- Singular value spread of one tracked layer

Expected Outcome

- SOTR ($\alpha \approx 0.5$) reaches target loss faster than AdamW
- More stable than Muon early in training

4.2 Mid-Scale Transformer (Critical Test)

Setup

- Model: ~300–500M parameter transformer
- Dataset: The Pile (subset)
- Same LR schedule across methods

Compare

- AdamW
- Muon
- Best SOTR configuration from speedrun

Metrics

- Final perplexity
- Training stability (divergence, NaNs)
- Gradient conditioning statistics

4.3 Ablations (Mandatory)

- Remove trust region ($\Delta = \infty$)
- Fix α, vary Δ
- Schedule $\alpha(t)$:
  - Linear warmup from 0 → 0.5 over first N steps
- Compare p = 1 vs p = 2 Newton–Schulz

4.4 Distributed Test (If Feasible)

- Use FSDP or DDP
- Apply SOTR **block-wise on local shards**
- Measure:
  - Throughput
  - Any cross-shard instability

---

5. Analysis & Interpretation (Must Be Explicit)

You must:

- Explain **why intermediate α works** (geometry + conditioning)
- Analyze trust-region effects vs standard grad clipping
- Identify regimes where:

    - SOTR ≈ Muon
    - SOTR ≈ SGD/Adam
- Discuss implications for:
    - Very deep transformers
    - Long-horizon pretraining

---

6. Deliverables

Produce:

1. **Clean PyTorch optimizer implementation**
2. **Experiment scripts & configs**
3. **Tables + plots**:
    - Loss vs steps
    - Condition number trends
4. **Final written conclusion** answering:

    > Does intermediate soft orthogonality outperform both extremes?

---

7. Success Criteria

This project is successful if **any $\alpha \in (0,1)$**:

- Matches Muon's conditioning benefits,
- Trains more stably or faster than AdamW,
- Requires less tuning than Muon.

If successful, conclude with:

- When SOTR should replace AdamW
- When full Muon is still preferable

Begin by implementing the optimizer, then run the small-scale speedrun immediately. Do not skip experiments.

Okay actually, why don't you take a shot at implementing this all?

Breakdown how you plan to do it, and then go ahead and implement it. The code you write should be suitable for an

undergrad coop to follow along, but also prioritize performance. Be sure to compare against muon, lion, muonbp, etc as appropriate

What do you need to know from me before getting started?

Here's how I'd tackle this end-to-end, in a way that (a) an undergrad can follow, (b) is performant enough to matter, and (c) cleanly supports comparisons vs **Muon / MuonBP / Lion / AdamW**.

I'll give you:

1. a concrete implementation plan
2. a working first implementation (optimizer + orthogonalization kernels + logging hooks)
3. an experiment harness scaffold that can plug into NanoGPT / a HF Trainer / your internal repo
4. what I need from you to wire it into *your* exact training stack and run "real" comparisons.

Plan (what I will build)

A) Core optimizer package (`optim_sotr/`)

- `sotr.py`: `SOTR` PyTorch optimizer
  - supports 2D params only (Linear/Conv reshaped); defaults to skipping non-2D tensors
  - supports:
    - `alpha` (soft-orthogonality blend)
    - `delta` (trust radius in Frobenius norm)
    - `ns_iters` (Newton–Schulz iterations; 1–2 recommended)
    - `beta1` momentum
    - optional `weight_decay` (decoupled like AdamW)
  - uses **batched operations** where possible; no SVD in the main path
- `orthogonal.py`: orthogonalization backends
  - `newton_schulz_orth(G, iters)` (fast approximate polar factor)
  - optional `svd_orth(G)` for correctness checks / ablations
  - handles rectangular matrices via transpose trick (more efficient) and stable scaling
- `utils.py`: helpers

- Frobenius norm, trust scaling, parameter filtering, lightweight stats

B) Baselines

- **Lion**: easy (pure PyTorch, no dependencies)
- **Muon**: depends what you mean by "Muon" exactly:
  - If you want **the Keller Jordan Muon reference** (blog/repo), I can mirror its update logic *once you tell me which exact implementation you want to match*.
  - In the meantime I'll include a "Muon-like" baseline: same orthogonalization backend + alpha=1 + no blending. (This is a good apples-to-apples baseline for SOTR's contribution.)
- **MuonBP**: typically means *block/partitioned orthogonalization* for scalability
  - I'll implement a generic **blockwise orthogonalization wrapper** that:
    - splits large matrices into blocks (e.g., 256×256 or 512×512)
    - orthogonalizes each block independently
    - recombines
  - This is comparable to MuonBP-style locality and is FSDP-friendly.

C) Experiment harness

- A minimal trainer that can run:
  - NanoGPT-style speedruns
  - HF causal LM (optional)
- Logging:
  - steps-to-target-loss
  - throughput
  - trust-region hit-rate (fraction of layers clipped)
  - singular-value spread proxy on a sampled layer (optional, computed occasionally)

What I need from you before I "fully" start wiring experiments

I can implement the optimizer + baselines immediately (below), but for the *full "compare against muon, lion, muonbp" in your environment*, I need 6 concrete things:

1. **Which training codebase?**
   - NanoGPT repo? HuggingFace Trainer? Lightning? Your internal stack?
2. **Which Muon / MuonBP reference should we match?**
   - Link or repo name (there are multiple "Muon" implementations floating around; small differences matter).
3. **Hardware + distribution**
   - single GPU vs multi-GPU? DDP? FSDP?

4. **Target model scale for tests**
    - e.g., 10–50M speedrun + 300–500M mid-scale
5. **Dataset choice / availability**
    - TinyShakespeare / OpenWebText subset / Pile subset / your preferred corpora
6. **Precision + compile stack**
    - fp16/bf16? torch.compile? fused optim? (matmul-heavy orthogonalization interacts with this)

If you reply with just (1) + (2), I can adapt everything else with sensible defaults.

Implementation: SOTR + orthogonalization + baselines

1) `orthogonal.py`

Fast approximate polar factor using Newton–Schulz. This is the “performance-critical” bit.

Python Run

```python
# orthogonal.py
import torch

def _fro_norm(x: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    # returns scalar tensor on same device/dtype
    return torch.sqrt(torch.sum(x * x) + eps)

@torch.no_grad()
def svd_orth(G: torch.Tensor) -> torch.Tensor:
    """
    Exact orthogonal factor O = U V^T from SVD(G) = U S V^T.
    Expensive; use for debugging/ablations only.
    """
    # Use float32 for stability if needed
    dtype_orig = G.dtype
    X = G.float() if G.dtype in (torch.float16, torch.bfloat16) else G
    U, _, Vh = torch.linalg.svd(X, full_matrices=False)
    O = U @ Vh
    return O.to(dtype_orig)

@torch.no_grad()
def newton_schulz_orth(G: torch.Tensor, iters: int = 2, eps: float = 1e-6) -> torch.Tensc
    """
    Approximate polar factor via Newton-Schulz iterations.

    Goal: compute something close to orthogonal matrix with singular values ~ 1,
    but with only 1-2 iterations (cheap).

    Works best when input is scaled so ||G||_F ~ 1.
    """
```

```
    assert G.ndim == 2
    dtype_orig = G.dtype

    # Use float32 for numerical stability in the iteration, then cast back.
    X = G.float() if G.dtype in (torch.float16, torch.bfloat16) else G

    # Scale to keep iteration stable (important!)
    # Common scaling: X / ||X||_F
    fro = _fro_norm(X, eps=1e-12)
    X = X / fro

    # Handle rectangular matrices efficiently:
    # If rows < cols, iterate on X X^T (smaller), else on X^T X
    # But classic NS for inverse sqrt can be applied directly with Y = X and Z = I
    # Here we use a common "polar NS" style iteration:
    # Y_{k+1} = 0.5 * Y_k * (3I - Y_k^T Y_k)
    # This pushes singular values toward 1.
    Y = X
    m, n = Y.shape
    I = torch.eye(n, device=Y.device, dtype=Y.dtype)

    # If n is huge, prefer forming Y Y^T if m < n:
    # Y_{k+1} = 0.5*(3Y - Y(Y^T Y)) is equivalent but can be arranged to reduce cost.
    # We do:
    #   if m < n: compute A = Y @ Y.T (m x m), then YTY = Y.T @ Y expensive.
    #   For simplicity/perf, choose branch:
    #   - if n <= m: use (Y.T @ Y) which is n x n
    #   - else: use (Y @ Y.T) which is m x m and rewrite update
    if n <= m:
        for _ in range(iters):
            YtY = Y.transpose(0, 1) @ Y            # (n,n)
            Y = 0.5 * (Y @ (3.0 * I - YtY))
    else:
        Im = torch.eye(m, device=Y.device, dtype=Y.dtype)
        for _ in range(iters):
            YYt = Y @ Y.transpose(0, 1)            # (m,m)
            # Rewrite: Y(3I - Y^T Y) = (3I - Y Y^T)Y
            Y = 0.5 * ((3.0 * Im - YYt) @ Y)

    # Y is approximately orthogonal direction; rescale is not needed for direction-only u
    return Y.to(dtype_orig)
```

2) `sotr.py`

This is the optimizer the undergrad can read and modify.

Python

Run

```
# sotr.py
import math
import torch
```

```python
from torch.optim.optimizer import Optimizer

from .orthogonal import newton_schulz_orth, svd_orth

def _fro_norm(x: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    return torch.sqrt(torch.sum(x * x) + eps)

def _is_matrix_param(p: torch.Tensor) -> bool:
    return p.ndim == 2

@torch.no_grad()
def _apply_trust_region(U: torch.Tensor, delta: float, eps: float = 1e-12):
    if delta is None or delta <= 0:
        return U, False
    nrm = _fro_norm(U, eps=eps)
    if nrm > delta:
        U = U * (delta / nrm)
        return U, True
    return U, False

class SOTR(Optimizer):
    """
    Soft-Orthogonal Trust Region Optimizer (SOTR)

    For each 2D parameter gradient G:
      O = approx_orth(G) via Newton-Schulz (few iters) or SVD (debug)
      U = alpha*O + (1-alpha)*normalize(G)
      U <- trust-region clip in Frobenius norm
      m <- beta1*m + (1-beta1)*U
      W <- W - lr*m   (with optional decoupled weight decay)
    """
    def __init__(
        self,
        params,
        lr: float = 1e-3,
        beta1: float = 0.9,
        alpha: float = 0.5,
        delta: float = 1.0,
        ns_iters: int = 2,
        eps: float = 1e-8,
        weight_decay: float = 0.0,
        decoupled_wd: bool = True,
        orth_backend: str = "ns",    # "ns" or "svd"
        block_size: int | None = None,  # if set, do blockwise orth on large matrices
        stats_every: int = 0,        # 0 disables
    ):
        if lr <= 0:
            raise ValueError("lr must be > 0")
        if not (0.0 <= alpha <= 1.0):
            raise ValueError("alpha must be in [0,1]")
        if not (0.0 <= beta1 < 1.0):
            raise ValueError("beta1 must be in [0,1)")
```

```
        if ns_iters < 0:
            raise ValueError("ns_iters must be >= 0")

        defaults = dict(
            lr=lr, beta1=beta1, alpha=alpha, delta=delta,
            ns_iters=ns_iters, eps=eps,
            weight_decay=weight_decay, decoupled_wd=decoupled_wd,
            orth_backend=orth_backend, block_size=block_size,
            stats_every=stats_every,
            step_count=0,
            trust_hits=0,
            trust_total=0,
        )
        super().__init__(params, defaults)

    @torch.no_grad()
    def _orth(self, G: torch.Tensor, group) -> torch.Tensor:
        backend = group["orth_backend"]
        ns_iters = group["ns_iters"]
        block_size = group["block_size"]

        def orth_one(X: torch.Tensor) -> torch.Tensor:
            if backend == "svd":
                return svd_orth(X)
            # default: NS
            if ns_iters == 0:
                # no orth step, acts like normalized gradient part only
                return X / (_fro_norm(X, eps=group["eps"]) + group["eps"])
            return newton_schulz_orth(X, iters=ns_iters, eps=group["eps"])

        if block_size is None:
            return orth_one(G)

        # Blockwise orth (MuonBP-like)
        m, n = G.shape
        bs = block_size
        out = torch.empty_like(G)
        for i in range(0, m, bs):
            i2 = min(i + bs, m)
            for j in range(0, n, bs):
                j2 = min(j + bs, n)
                out[i:i2, j:j2] = orth_one(G[i:i2, j:j2])
        return out

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
```

```
            lr = group["lr"]
            beta1 = group["beta1"]
            alpha = group["alpha"]
            delta = group["delta"]
            eps = group["eps"]
            wd = group["weight_decay"]
            decoupled_wd = group["decoupled_wd"]

            group["step_count"] += 1
            step_idx = group["step_count"]

            for p in group["params"]:
                if p.grad is None:
                    continue
                if not _is_matrix_param(p):
                    # For now: skip non-2D params.
                    # Option: you can extend this to 1D biases/LayerNorm with AdamW or Li
                    continue

                G = p.grad
                if G.is_sparse:
                    raise RuntimeError("SOTR does not support sparse gradients.")

                # Optional decoupled weight decay
                if wd != 0.0 and decoupled_wd:
                    p.mul_(1.0 - lr * wd)

                # 1) Orthogonalize (approx)
                O = self._orth(G, group)

                # 2) Blend with normalized gradient
                Gn = G / (_fro_norm(G, eps=eps) + eps)
                U = alpha * O + (1.0 - alpha) * Gn

                # 3) Trust-region (Frobenius cap)
                U, clipped = _apply_trust_region(U, delta=delta, eps=eps)
                group["trust_total"] += 1
                if clipped:
                    group["trust_hits"] += 1

                # 4) Momentum
                state = self.state[p]
                if "m" not in state:
                    state["m"] = torch.zeros_like(p)

                m = state["m"]
                m.mul_(beta1).add_(U, alpha=(1.0 - beta1))

                # 5) Update
                p.add_(m, alpha=-lr)

                # Optional coupled (L2) weight decay (rarely preferred)
```

```
                if wd != 0.0 and (not decoupled_wd):
                    p.add_(p, alpha=-lr * wd)

        return loss

    @torch.no_grad()
    def get_trust_region_stats(self):
        # aggregated across param groups
        hits = 0
        total = 0
        for group in self.param_groups:
            hits += int(group.get("trust_hits", 0))
            total += int(group.get("trust_total", 0))
        rate = hits / max(1, total)
        return {"trust_hits": hits, "trust_total": total, "trust_rate": rate}
```

3) `lion.py` baseline

A clean Lion implementation for comparison.

Python Run

```python
# lion.py
import torch
from torch.optim.optimizer import Optimizer

class Lion(Optimizer):
    """
    Lion optimizer (sign-based momentum update).
    This is a common baseline in LLM training.

    Update:
      m = beta1*m + (1-beta1)*g
      p -= lr * sign(m)
    with optional decoupled weight decay.
    """
    def __init__(self, params, lr=1e-4, beta1=0.9, beta2=0.99, weight_decay=0.0):
        defaults = dict(lr=lr, beta1=beta1, beta2=beta2, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
            lr = group["lr"]
            beta1 = group["beta1"]
            beta2 = group["beta2"]
```

```
            wd = group["weight_decay"]

            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad

                if wd != 0.0:
                    p.mul_(1.0 - lr * wd)

                state = self.state[p]
                if "m" not in state:
                    state["m"] = torch.zeros_like(p)

                m = state["m"]
                # Lion uses two betas: first for update direction, second for momentum tr
                update = m.mul(beta1).add(g, alpha=1 - beta1)
                p.add_(torch.sign(update), alpha=-lr)
                m.mul_(beta2).add_(g, alpha=1 - beta2)

        return loss
```

4) “Muon-like” baseline

This is *exactly* SOTR with `alpha=1` (and your chosen orth backend/iters). That’s useful because it isolates the “softness + trust region” effect.

Python Run

```python
# muon_like.py
from .sotr import SOTR

def MuonLike(params, lr=1e-3, beta1=0.9, ns_iters=5, delta=1.0, weight_decay=0.0, block_s
    # alpha=1 => pure orth step; this approximates Muon behavior depending on orth backen
    return SOTR(
        params,
        lr=lr, beta1=beta1,
        alpha=1.0, delta=delta,
        ns_iters=ns_iters,
        weight_decay=weight_decay,
        orth_backend="ns",
        block_size=block_size,
    )
```

5) Minimal usage example (trainer-agnostic)

This is the glue pattern you’ll use in NanoGPT/HF/etc.

Python Run

```python
# example_usage.py
import torch
from optim_sotr.sotr import SOTR
from optim_sotr.lion import Lion
from optim_sotr.muon_like import MuonLike

def build_optimizer(name, model, lr):
    params = [p for p in model.parameters() if p.requires_grad]

    if name == "sotr":
        return SOTR(params, lr=lr, alpha=0.5, delta=1.0, ns_iters=2, beta1=0.9, weight_de
    if name == "sotr_bp":
        return SOTR(params, lr=lr, alpha=0.5, delta=1.0, ns_iters=2, beta1=0.9, weight_de
    if name == "muon_like":
        return MuonLike(params, lr=lr, ns_iters=5, delta=1.0, beta1=0.9, weight_decay=0.0
    if name == "lion":
        return Lion(params, lr=lr, beta1=0.9, beta2=0.99, weight_decay=0.01)
    raise ValueError(f"Unknown optimizer {name}")
```

What's missing (and how we'll fill it once you answer)

To truly "compare against muon, muonbp, etc as appropriate", we need to align definitions:

- **Muon**: is it *exact SVD UV^T each step*, or *Newton–Schulz polar factor* with specific coefficients/normalizations?
- **MuonBP**: what block size / partition scheme / communication pattern do you want to match?

I've already implemented the two main knobs you need to replicate most variants:

- `ns_iters` controls orth strength/cost
- `block_size` gives you MuonBP-like scalability

But if you want *exact* apples-to-apples vs a specific repo's "MuonBP", I need the reference.

My "minimum needed from you" (so I can proceed without ambiguity)

Reply with:

1. **Training codebase**: (NanoGPT / HF / Lightning / custom) + link/name
2. **Which Muon + MuonBP implementation** you mean (repo URL or file)
3. Target speedrun config:
    - model size (e.g., NanoGPT default)
    - dataset (Tiny Shakespeare vs OWT vs pile subset)
4. Hardware: single GPU or multi-GPU (and whether you'll use FSDP)

That's it.

Once you send that, I'll:

- adapt this into your repo layout,
- add the exact baseline implementations matching your references,
- write the full experiment scripts + logging + ablations exactly as you outlined (α sweep, Δ sweep, with/without trust, schedule α(t), ns_iters 1 vs 2).

Baselines incl lion, Muon should be the repo version, but also include a muon like, and ignore muonbp. Also compare against AdamW.

Nanogpt repo is good. Use whatever was used in the muon and lion papers that would make the best comparison for good evaluations. For hardware and distribution, do what has been done in Dion, Muon, lion, etc.

The rest is up to you based on prompt I gave.

Got it. With your constraints:

- **Baselines:** AdamW, Lion (official algo), Muon (**KellerJordan/Muon repo implementation**), plus a **Muon-like** (our own minimal "Muon update rule" using the published Newton–Schulz-5 polynomial, no aux-Adam wrapper), and **SOTR** (your method).
- **Ignore MuonBP**.
- **Repo:** NanoGPT.
- **Distributed/hardware:** we'll mirror what the NanoGPT speedrun + Muon usage patterns do (DDP via `torchrun`; BF16; optional `torch.compile`), which is consistent with how these optimizers are typically evaluated in practice.

Below: (1) how I'll implement + wire it into NanoGPT, (2) the actual code (drop-in files + `train.py` patch), (3) runnable commands & evaluation protocol.

What I'll implement (breakdown)

1) Optimizer implementations (new folder `optimizers/`)

- `optimizers/lion_official.py`: Lion matching the reference implementation behavior (betas (0.9, 0.99), sign of interpolated momentum, decoupled WD).
- `optimizers/sotr.py`: SOTR for 2D params:
  - orth step = **Muon's Newton–Schulz polynomial iteration** but with `ns_steps=1-2` (or configurable). Muon's writeup provides the tuned coefficients + normalization/transposition trick.
  - blend with normalized grad via `alpha`
  - Frobenius trust region cap via `delta`
  - momentum `beta1` (SGD-momentum style)
- `optimizers/muon_like.py`: Muon-like baseline:
  - same NS orth transform as Muon writeup (default 5 steps), **alpha=1**, no blending, no aux-adam.

2) Muon repo baseline (installed dependency)

We'll use the **repo version** as requested: `pip install git+https://github.com/KellerJordan/Muon`, then import `MuonWithAuxAdam` and create param groups like the repo README recommends.

3) NanoGPT integration

Patch `train.py` to accept an `optimizer` name:

- `"adamw"` (existing)
- `"lion"`
- `"muon"` (repo)
- `"muon_like"`
- `"sotr"`

And build param groups:

- "Hidden weights" = transformer block matrices (linear weights) (ndim==2) inside `transformer.h.*`
- "Nonhidden / embeddings / head / norms / biases" = everything else (AdamW for Muon baseline; for Lion baseline it's okay to run Lion across all params, but we'll also provide "Lion+AdamW split" option if you want apples-to-apples with Muon's split philosophy).

Muon's recommended usage is to apply Muon only to hidden 2D weights and use AdamW for the rest.

Code: drop-in files for NanoGPT

Create a new directory at repo root:

```
nanoGPT/
  optimizers/
    __init__.py
    lion_official.py
    sotr.py
    muon_like.py
```

`optimizers/lion_official.py`

Python Run

```python
import torch
from torch.optim.optimizer import Optimizer

class Lion(Optimizer):
    """
    Lion optimizer (official behavior): betas=(0.9, 0.99), decoupled weight decay,
    update is sign(beta1*m + (1-beta1)*grad), then m <- beta2*m + (1-beta2)*grad.

    This matches the widely-circulated reference implementation behavior. :contentReferen
    """
    def __init__(self, params, lr=1e-4, betas=(0.9, 0.99), weight_decay=0.0):
        if lr <= 0:
            raise ValueError("lr must be > 0")
        b1, b2 = betas
        if not (0.0 <= b1 < 1.0 and 0.0 <= b2 < 1.0):
            raise ValueError("betas must be in [0,1)")
        if weight_decay < 0:
            raise ValueError("weight_decay must be >= 0")

        defaults = dict(lr=lr, betas=betas, weight_decay=weight_decay)
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
            lr = group["lr"]
            beta1, beta2 = group["betas"]
            wd = group["weight_decay"]

            for p in group["params"]:
                if p.grad is None:
                    continue
```

```
                g = p.grad
                if g.is_sparse:
                    raise RuntimeError("Lion does not support sparse gradients.")

                # decoupled weight decay
                if wd != 0.0:
                    p.mul_(1.0 - lr * wd)

                state = self.state[p]
                if "exp_avg" not in state:
                    state["exp_avg"] = torch.zeros_like(p)

                m = state["exp_avg"]

                # update direction
                update = m.mul(beta1).add(g, alpha=(1.0 - beta1)).sign_()
                p.add_(update, alpha=-lr)

                # momentum update
                m.mul_(beta2).add_(g, alpha=(1.0 - beta2))

        return loss
```

optimizers/sotr.py

Python

Run

```
import torch
from torch.optim.optimizer import Optimizer

def fro_norm(x: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    return torch.sqrt(torch.sum(x * x) + eps)

@torch.no_grad()
def muon_newton_schulz(G: torch.Tensor, steps: int, eps: float = 1e-7) -> torch.Tensor:
    """
    Muon-style Newton-Schulz polynomial iteration as published in the Muon writeup,
    but with configurable step count. :contentReference[oaicite:6]{index=6}

    Uses the tuned coefficients (a,b,c) and:
      X /= (||X|| + eps)
      transpose trick for rectangular matrices.
    """
    assert G.ndim == 2
    a, b, c = (3.4445, -4.7750, 2.0315)

    # Muon uses bf16 internally for speed, but normalizes for stability.
    X = G.to(dtype=torch.bfloat16)
    X = X / (fro_norm(X, eps=eps) + eps)

    transposed = False
```

```
    if G.size(0) > G.size(1):
        X = X.t()
        transposed = True

    # Iterate: A = X X^T ; B = bA + cA^2 ; X = aX + B X
    for _ in range(steps):
        A = X @ X.t()
        B = b * A + c * (A @ A)
        X = a * X + (B @ X)

    if transposed:
        X = X.t()

    return X.to(dtype=G.dtype)

class SOTR(Optimizer):
    """
    Soft-Orthogonal Trust Region optimizer.

    For each 2D parameter gradient G:
      O = muon_newton_schulz(G, ns_steps)
      U = alpha*O + (1-alpha)*G/||G||_F
      if ||U||_F > delta: U <- U * delta/||U||_F
      m <- beta1*m + (1-beta1)*U
      W <- W - lr*m
    """
    def __init__(
        self,
        params,
        lr: float = 1e-3,
        beta1: float = 0.9,
        alpha: float = 0.5,
        delta: float = 1.0,
        ns_steps: int = 2,
        eps: float = 1e-8,
        weight_decay: float = 0.0,   # optional decoupled wd
        stats: bool = True,
    ):
        if lr <= 0:
            raise ValueError("lr must be > 0")
        if not (0.0 <= beta1 < 1.0):
            raise ValueError("beta1 must be in [0,1)")
        if not (0.0 <= alpha <= 1.0):
            raise ValueError("alpha must be in [0,1]")
        if ns_steps < 0:
            raise ValueError("ns_steps must be >= 0")
        if delta is not None and delta <= 0:
            raise ValueError("delta must be > 0 (or None)")

        defaults = dict(
            lr=lr, beta1=beta1, alpha=alpha, delta=delta,
            ns_steps=ns_steps, eps=eps, weight_decay=weight_decay,
```

```
            trust_hits=0, trust_total=0, stats=stats,
        )
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()

        for group in self.param_groups:
            lr = group["lr"]
            beta1 = group["beta1"]
            alpha = group["alpha"]
            delta = group["delta"]
            ns_steps = group["ns_steps"]
            eps = group["eps"]
            wd = group["weight_decay"]

            for p in group["params"]:
                if p.grad is None:
                    continue
                if p.grad.is_sparse:
                    raise RuntimeError("SOTR does not support sparse gradients.")

                # Optional: decoupled weight decay
                if wd != 0.0:
                    p.mul_(1.0 - lr * wd)

                g = p.grad

                # Apply only to matrices; skip vectors/scalars.
                if p.ndim != 2:
                    continue

                # Orth step
                if ns_steps > 0:
                    O = muon_newton_schulz(g, steps=ns_steps, eps=1e-7)
                else:
                    O = g / (fro_norm(g, eps=eps) + eps)

                # Blend with normalized grad
                Gn = g / (fro_norm(g, eps=eps) + eps)
                U = alpha * O + (1.0 - alpha) * Gn

                # Trust region (Frobenius cap)
                if delta is not None:
                    nrm = fro_norm(U, eps=eps)
                    group["trust_total"] += 1
                    if nrm > delta:
                        U = U * (delta / nrm)
```

```
                        group["trust_hits"] += 1

                # Momentum
                state = self.state[p]
                if "m" not in state:
                    state["m"] = torch.zeros_like(p)

                m = state["m"]
                m.mul_(beta1).add_(U, alpha=(1.0 - beta1))

                # Update
                p.add_(m, alpha=-lr)

        return loss

    @torch.no_grad()
    def trust_region_stats(self):
        hits, total = 0, 0
        for group in self.param_groups:
            hits += int(group.get("trust_hits", 0))
            total += int(group.get("trust_total", 0))
        return dict(trust_hits=hits, trust_total=total, trust_rate=hits / max(1, total))
```

optimizers/muon_like.py

Python Run

```
import torch
from torch.optim.optimizer import Optimizer
from .sotr import muon_newton_schulz

class MuonLike(Optimizer):
    """
    Minimal Muon-like baseline:
    - momentum on the matrix update
    - orthogonalize via Muon NS polynomial (default 5 steps)
    - no aux AdamW wrapper (unlike repo MuonWithAuxAdam)
    This isolates the orth+momentum effect for apples-to-apples with SOTR.
    """
    def __init__(self, params, lr=0.02, beta1=0.95, ns_steps=5, eps=1e-8, weight_decay=0
        if lr <= 0:
            raise ValueError("lr must be > 0")
        defaults = dict(lr=lr, beta1=beta1, ns_steps=ns_steps, eps=eps, weight_decay=wei
        super().__init__(params, defaults)

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
```

```
        for group in self.param_groups:
            lr = group["lr"]
            beta1 = group["beta1"]
            ns_steps = group["ns_steps"]
            wd = group["weight_decay"]

            for p in group["params"]:
                if p.grad is None:
                    continue
                if p.ndim != 2:
                    continue

                if wd != 0.0:
                    p.mul_(1.0 - lr * wd)

                g = p.grad
                U = muon_newton_schulz(g, steps=ns_steps)

                state = self.state[p]
                if "m" not in state:
                    state["m"] = torch.zeros_like(p)

                m = state["m"]
                m.mul_(beta1).add_(U, alpha=(1.0 - beta1))
                p.add_(m, alpha=-lr)

        return loss
```

`optimizers/__init__.py`

Python Run

```
from .lion_official import Lion
from .sotr import SOTR
from .muon_like import MuonLike
```

Patch NanoGPT `train.py` to select optimizers

NanoGPT's `train.py` defines default configs and constructs AdamW by default.

You'll patch the optimizer creation section (search for `optimizer = torch.optim.AdamW(...)` and/or `configure_optimizers` depending on your NanoGPT fork).

Add near the top-level config options:

Python Run

```
optimizer_name = 'adamw'  # 'adamw' | 'lion' | 'muon' | 'muon_like' | 'sotr'
```

Then in the place where optimizer is constructed, replace with something like this:

Python Run

```
import torch
from optimizers import Lion, SOTR, MuonLike

def split_params_for_muon(model):
    """
    Muon guidance: use Muon on hidden 2D weights, AdamW on embeddings, head, and 1D param
    For NanoGPT model.py, typical names:
      transformer.wte, transformer.wpe, lm_head are embeddings/head.
      transformer.h.* are the blocks.
    """
    hidden_weights = []
    other_params = []

    for name, p in model.named_parameters():
        if not p.requires_grad:
            continue

        is_embed_or_head = (
            name.startswith("transformer.wte")
            or name.startswith("transformer.wpe")
            or name.startswith("lm_head")
        )

        if (p.ndim >= 2) and (not is_embed_or_head) and name.startswith("transformer.h"):
            hidden_weights.append(p)
        else:
            other_params.append(p)

    return hidden_weights, other_params

# ---- build optimizer ----
if optimizer_name == "adamw":
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=learning_rate,
        betas=(beta1, beta2),
        weight_decay=weight_decay,
    )

elif optimizer_name == "lion":
    optimizer = Lion(
        model.parameters(),
        lr=learning_rate,
```

```
        betas=(0.9, 0.99),
        weight_decay=weight_decay,
    )

elif optimizer_name == "muon":
    # repo version
    # pip install git+https://github.com/KellerJordan/Muon :contentReference[oaicite:9]{i
    from muon import MuonWithAuxAdam

    hidden_weights, other_params = split_params_for_muon(model)

    param_groups = [
        dict(params=hidden_weights, use_muon=True, lr=0.02, weight_decay=weight_decay),
        dict(params=other_params, use_muon=False, lr=learning_rate, betas=(beta1, beta2),
    ]
    optimizer = MuonWithAuxAdam(param_groups)

elif optimizer_name == "muon_like":
    hidden_weights, other_params = split_params_for_muon(model)

    # MuonLike only applies to 2D hidden weights; others use AdamW for fairness.
    opt_hidden = MuonLike(hidden_weights, lr=0.02, beta1=0.95, ns_steps=5, weight_decay=
    opt_other = torch.optim.AdamW(other_params, lr=learning_rate, betas=(beta1, beta2), w

    optimizer = (opt_hidden, opt_other)  # see note below

elif optimizer_name == "sotr":
    hidden_weights, other_params = split_params_for_muon(model)

    opt_hidden = SOTR(
        hidden_weights,
        lr=0.02,
        beta1=0.95,
        alpha=0.5,
        delta=1.0,
        ns_steps=2,
        weight_decay=weight_decay,
    )
    opt_other = torch.optim.AdamW(other_params, lr=learning_rate, betas=(beta1, beta2), w

    optimizer = (opt_hidden, opt_other)  # see note below

else:
    raise ValueError(f"Unknown optimizer_name: {optimizer_name}")
```

Important note: NanoGPT expects a single optimizer object. For the tuple `(opt_hidden, opt_other)` approach, you'll update the training loop to call `step()` and `zero_grad()` on both. That patch is small:

- Replace `optimizer.zero_grad(set_to_none=True)` with:

```python
if isinstance(optimizer, tuple):
    for opt in optimizer:
        opt.zero_grad(set_to_none=True)
else:
    optimizer.zero_grad(set_to_none=True)
```

- Replace `optimizer.step()` with:

```python
if isinstance(optimizer, tuple):
    for opt in optimizer:
        opt.step()
else:
    optimizer.step()
```

This keeps the code extremely readable for a co-op student, and avoids fighting the param-group semantics between “matrix-only optimizer” and AdamW.

Experiment protocol (matches your earlier prompt)

Small-scale speedrun (cheap sanity + α sweep)

Run NanoGPT Shakespeare char or a small OpenWebText subset, compare:

- AdamW
- Lion
- Muon (repo baseline)
- MuonLike
- SOTR with α ∈ {0.3, 0.5, 0.7, 1.0} and ns_steps ∈ {1,2}

Track:

- steps/time to target loss
- `trust_rate` from `SOTR.trust_region_stats()` (optional print every N iters)
- occasional singular value spread on one layer (compute every ~1000 steps to avoid overhead)

Mid-scale (the real check)

Use NanoGPT’s default OpenWebText GPT-2-ish config and run short training (same tokens) for:

- AdamW
- Lion

- Muon (repo)
- Best SOTR

This is exactly the kind of comparison shown/discussed in Muon's writeup (Muon vs AdamW and other strong optimizers).

Distributed

Use NanoGPT's DDP path (`torchrun --nproc_per_node=8 train.py ...`). That's the closest practical analogue to the way speedruns are actually done with these optimizers.

Commands (examples)

Install deps

Bash

```
pip install -r requirements.txt
pip install git+https://github.com/KellerJordan/Muon
```

(Muon install is per the repo.)

Single GPU speedrun example

Bash

```
python train.py config/train_shakespeare_char.py \
  --optimizer_name=sotr --compile=True
```

8-GPU DDP example

Bash

```
torchrun --standalone --nproc_per_node=8 train.py config/train_openwebtext.py \
  --optimizer_name=muon --compile=True --dtype=bfloat16
```

One last thing I *do* need from you (minimal)

You said "rest is up to you," which is fine, but I still need **one concrete choice** to avoid guessing wrong:

1. **Which NanoGPT dataset path are you actually using?**

- `shakespeare_char`, `shakespeare`, `openwebtext`, or your own?

(This aff… files I target and what "good evaluation" means.)

Sources

Write a chatgpt deep research prompt to review the latest literature on this topic area.

Here's a **copy-paste "Deep Research" prompt** you can give to ChatGPT (or any research-mode agent) to review the **latest literature** around **soft orthogonalization / Muon-style optimizers / trust-region constraints for deep nets**—and to specifically situate **SOTR** in that landscape.

Deep Research Prompt: Soft Orthogonalization + Trust-Region Optimizers for Deep Nets

You are a research analyst conducting a **literature review of the most recent work (prioritize 2023–present, but include seminal older work)** on optimizers and update transformations that:

- enforce **(soft) orthogonality / near-isometry** in updates or weights,
- use **polar decomposition / orthogonalization approximations** (Newton–Schulz, SVD, QR, Cayley, etc.),
- incorporate **trust-region** or **matrix-norm constraints** on updates,
- improve conditioning and stability in **very deep neural networks / transformers**.

Your goal is to produce a **structured, citation-heavy report** that would help evaluate and refine a proposed optimizer idea:

> **Soft-Orthogonal Trust Region (SOTR):** for each 2D gradient matrix G, compute a partial or approximate orthogonal factor O (e.g., 1–2 Newton–Schulz steps toward a polar factor), blend with normalized gradient $G/\|G\|_F$ via α, then enforce a Frobenius-norm trust region $\|U\|_F \le \Delta$, apply momentum, and update weights.

What you must do

1) Define the topic map

Create a taxonomy of relevant literature into buckets such as:

- Orthogonal/near-orthogonal **update transforms** (polar, NS, QR, Cayley, etc.)

- Orthogonal/near-orthogonal **weight parameterizations** (Stiefel manifold optimization, orthogonal retractions)
- Optimizers that use **normalization/sign** updates (e.g., normalized gradient, signSGD family, Lion-like)
- **Trust-region** methods adapted to deep learning (TRPO, KFAC trust regions, SAM-like neighborhoods, clipping variants)
- Conditioning / spectrum shaping (preconditioning, Shampoo/KFAC, spectral regularization, gradient whitening)
- Large-scale LLM training stability methods (clipping, scaling laws, optimizer comparisons)

For each bucket, list key papers and what problem they solve.

2) Find and summarize the *latest* literature
Search the web (papers, arXiv, blogs from credible authors, major labs) and prioritize:

- recent papers (2023–present),
- anything directly referencing **Muon** or "polar / orthogonalized updates,"
- work connecting **matrix constraints** and **trust-region-like bounds** in large models.

For each relevant item, provide:

- full citation + link
- short summary (2–6 bullets)
- what it contributes (mechanism)
- what assumptions it relies on (scale, architecture, compute)
- how it relates to SOTR (supporting, competing, or contradicting)

3) Identify the closest conceptual neighbors to SOTR
Explicitly compare SOTR to the nearest prior art:

- Is blending $\alpha O + (1-\alpha)\hat{G}$ similar to any known interpolation schemes?
- Are there precedents for "soft projection" toward a manifold vs hard projection?
- Are Frobenius trust regions used per-matrix (not global clipping) anywhere?

Make a table:

- **Method**
- **What is constrained / normalized**
- **How (hard vs soft)**
- **Compute cost**
- **Known results (transformers?)**
- **Predicted behavior vs SOTR**

4) Extract actionable design lessons
From the literature, distill:

- when orthogonalization helps (early vs late training, depth, scale),
- what failure modes appear (instability, slower convergence, losing magnitude signal),
- how trust-region ideas in deep learning are typically implemented and tuned.

Then propose concrete refinements to SOTR:

- scheduling strategies for $\alpha(t)$ and/or $\Delta(t)$
- layerwise Δ calibration rules (dimension-scaled)
- whether to use Frobenius vs spectral norm constraints
- whether momentum should be applied before vs after blending/orth step
- whether to apply SOTR only to certain parameter subsets (e.g., attention/projection matrices)

5) Give a practical "evaluation blueprint"
Using best practices from the literature, propose:

- minimal set of baselines (AdamW, Lion, Muon, Shampoo/KFAC-family if relevant)
- fair hyperparameter matching rules
- what metrics to log (loss vs wall time, grad/weight spectral stats, trust-region hit-rate, stability incidents)
- what small-to-mid scale experiments are most diagnostic for SOTR's claims

6) Output format requirements
Produce:

1. **Executive summary** (10–15 bullets, high signal)
2. **Topic map + key references** per bucket
3. **Annotated bibliography** (most relevant 15–30 items)
4. **Comparison table** (SOTR vs neighbors)
5. **Design implications** (what to copy, what to avoid)
6. **Experiment plan** (fastest credible validation path)
7. **Open questions** (what evidence is missing)

Quality bar

- Prefer peer-reviewed papers, widely cited arXiv, and reputable lab blogs.
- When using blogs, cross-check against primary sources.
- Be explicit about uncertainty and conflicting evidence.
- Include citations for every nontrivial factual claim.

Start now by performing web searches for the most relevant recent papers and Muon-related references, then build the report.

If you want, I can also tailor that prompt to a specific research style (e.g., “focus on LLM training at 300M–7B,” “focus on theory/geometry,” or “focus on implementation details and kernels”).